

Validate the decision, then ask for history

Daily learning note | 29 September 2026 | Q006b

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a bounded calendar validate the intended pre-open research decision while rejecting closed dates, wrong clocks and unknown history? The hypothesis is that decision validity, scheduled core time and sufficient feature history are separate checks. Combining them would either reject legitimate decisions or admit incomplete inputs. Yesterday's Q006a supplied the reviewed contract and reference schedule; today implements that contract without expanding its dates or claiming production exchange coverage.

Beginner explanation and definitions

A shop can hold a planning meeting before opening. Knowing that the meeting is correctly scheduled still does not prove that twenty earlier daily sales reports exist. Research has the same separation: the decision clock, trading clock and information history answer different questions.

Session: a date with scheduled core trading. **Decision:** the instant at which a policy selects information. **UTC instant:** a timezone-independent point on the clock. **Known closure:** an explicit holiday or weekend inside reviewed coverage. **Unknown coverage:** a date for which the fixture offers no evidence. **Half-open interval:** includes its beginning and excludes its end. **Lookback:** a fixed number of earlier sessions. **Checksum:** a fingerprint of exact file bytes, not proof of truth.

A closed day is a known answer. A date beyond the reference is an unanswered question. Returning False for both would conceal the missing evidence. The new API therefore returns None for a known closed session date and raises CalendarCoverageError for dates or history it cannot establish.

Data, metrics and predeclared protocol

The unchanged Q006a fixture covers 29 October through 30 November 2026: 33 explicit dates and 22 scheduled NYSE cash-equity sessions. Calendar facts are prospective reference facts; the submitted clock times are synthetic test cases. No market prices, actual trades or investment returns are used.

Before implementation, acceptance required all intended 09:00 decisions, holiday/weekend rejection, UTC conversion, early closes, exact boundaries, unknown-date errors and complete prior-session windows. The experiment checks all 22 sessions, six core-boundary cases per session and nine invalid decisions. Tests also use hand-worked UTC examples, an explicit 20-session date list, malformed inputs, changed fixture bytes and two independent subprocess replays. No empirical hypothesis is tested today.

Mathematics and worked example

For scheduled session s , let $D(s)$, $O(s)$, $C(s)$ denote decision, core open and core close. The policy is $D(s) < O(s) < C(s)$. Decision validity requires $t = D(s)$; scheduled core time requires $O(s) \leq t < C(s)$. A decision can therefore be valid and outside core time. Neither statement establishes an executable fill. The exact close is excluded by this project convention; closing auctions require a separate model.

Let $H_n(s)$ be the final n listed session dates strictly before s , in ascending order. Return $H_n(s)$ only if n is a positive integer and at least n prior sessions exist within coverage. Otherwise raise an explicit error. Never substitute calendar days, create a missing bar or return a shorter window.

Decision date	Decision UTC	Prior 20-session window	Result
25 November	14:00	Only 19 prior sessions known	Insufficient history
27 November	14:00	29 October through 25 November	Exactly 20 sessions
30 November	14:00	30 October through 27 November	Exactly 20 sessions

These endpoints enclose only listed sessions. November 26 is a holiday, so November 27's immediate predecessor is November 25. November 27 closes at 13:00 New York, or 18:00 UTC. Its scheduled core interval is [14:30, 18:00) UTC and lasts 210 minutes. At 17:59:59.999999 the predicate is True; at 18:00:00 it is False. The valid decision at 14:00 precedes the open by 30 minutes.

Before the autumn clock change, 09:00 New York corresponds to 13:00 UTC. Afterwards it corresponds to 14:00 UTC. The conversion is $UTC = \text{wall time} - \text{UTC offset}$. A fixed 13:00 UTC rule becomes 08:00 New York on November 2 and is rejected. Equivalent UTC, New York, Tokyo and +14:00 representations of the correct instant are accepted, even if their displayed calendar dates differ.

Primary sources consulted

NYSE's official schedule confirms the 2026 Thanksgiving closure, next-day 13:00 close and regular 09:30 to 16:00 core session. This supports the unchanged fixture, not a claim that future trading occurred. <https://www.nyse.com/trade/hours-calendars>

Python documents aware versus naive datetimes, UTC conversion and IANA zone support. Timestamp inputs are normalized through UTC before the New York date is selected; original raw-clock validation remains Q005's responsibility. The installed rules are checked against every pinned fixture timestamp at load. <https://docs.python.org/3/library/datetime.html> <https://docs.python.org/3/library/zoneinfo.html>

Sources consulted on 29 September 2026. The fixture retains its original September 28 source edition and separate timezone provenance. No exchange release ID, source archive or complete environment pin has been invented. A source or timezone correction requires investigation and a reviewed version change.

Limitations

This is a prospective 33-date teaching calendar. It does not cover emergency closures, halts, auctions, other venues or the intended 2010-2025 empirical study. A checksum establishes fixture identity only. Loading verifies this fixture's relevant offsets, not a fully pinned timezone database. Lookup is linear in 22 sessions; no performance claim is made. History completeness means scheduled dates exist, not that observations exist or were received in time. Q001 availability remains a separate requirement.

Implementation and reproduction

`src/quant_research/data/calendar.py` exposes immutable `Session` values and `NyseAutumn2026Calendar`. Pass the existing fixture path to its constructor. The loader requires the reviewed file's exact SHA-256 and checks the installed timezone conversions; it is deliberately limited to this one version. Even a formatting-only byte change fails. This is a bounded loader, not a general JSON calendar parser.

`session_on` takes a New York date; datetime values are rejected to prevent ambiguous date labelling. `validate_decision` accepts an aware instant and returns its session only at exactly the policy clock. `is_core_time` separately checks scheduled hours. `previous_sessions` returns a complete immutable tuple or raises. The first date can have a valid decision while failing a one-session history request.

Run `make calendar-experiment` to regenerate `experiments/calendar/decision-validation.json`. Use `.venv/bin/python scripts/calendar_experiment.py --output /tmp/calendar-replay.json` for a scratch replay. Run `make check` for all tests and quality checks; `make calendar-audit` and `make manifest-experiment` retain earlier evidence. Run `make note NOTE_DATE=2026-09-29` to regenerate this teaching PDF.

Observed results and validation

Check	Observed result
Valid pre-open decisions	All 22 accepted; a core-only rule would reject all 22
Core boundaries	All 132 open/close and adjacent-microsecond checks pass
Invalid diagnostic decisions	All 9 rejected, including naive and out-of-range cases
History	November 25 fails; November 27 and 30 each return 20 prior sessions
Replay	Two subprocess results match archived JSON bytes exactly
Python suite	71 new tests; 363 total pass
Code quality	Ruff lint; 35 formatted files; strict mypy on 17 package files pass
C++ and environment	Fresh warning-clean C++17 build, golden output and pip check pass
Prior provenance	Four original Q003 source hashes still verify

The initial targeted tests and type checks passed. Lint found three overlong diagnostic strings; wrapping those literals fixed the issue without changing results. The subsequent complete `make check` passed. No failed check was suppressed, dependency added or coverage percentage inferred.

Tests additionally prove all 11 hand-listed closures, incorrect fixed UTC hours, local versus UTC date boundaries, exact history ordering, nonpositive/noninteger counts, immutable outputs, modified fixture rejection and simulated timezone-rule mismatch rejection. A holiday publication still passes generic ingestion: its availability timestamp is valid even while the exchange is closed.

Interview explanation and practical lesson

Explain the three checks in order: identify the session from the New York date, verify the exact research clock, then request the complete prior-session window. Keep scheduled trading hours separate. A valid 09:00 decision cannot authorize a 09:00 fill, and twenty known dates cannot prove twenty known prices. Use November 25 versus November 27 to show why valid decisions can differ in history readiness.

For boundary questions, state the interval convention before coding. Demonstrate the exact open, the final microsecond before the close and the exact close. Normalize instants before comparison; never guess a timezone for naive input. Distinguish known closed dates from unknown coverage.

These are examples of validation and data-engineering reasoning, not evidence of trading performance or independently acquired personal expertise. Reproduce the example before using it as an interview claim.

Next work and necessary input

Q006b and its Q006 parent are complete within their bounded acceptance criteria. Wednesday September 30 advances C002: reusable C++ savings growth with explicit compounding and invalid-input checks. The next non-C++ task is Q008 synthetic corporate-action arithmetic on October 1. Q004a still needs actual lawful source access and sample evidence; Q005b still tracks provenance and multi-batch integration.

No input is needed for the next engineering task. For empirical data work, record existing entitlement or a source/budget preference without sending credentials. Personal review remains Not yet reviewed; publication and routine progress remain authorised. New programme research stops after December 31.

Check your understanding

1. Why should a calendar raise for December 1 instead of calling it a closed day?
2. Can the first covered session have a valid decision but insufficient feature history?
3. What happens exactly at the early close, and does this settle closing-auction execution?
4. Why can a timestamp displaying the next date in another timezone still represent a valid decision?
5. Do twenty prior sessions prove that a 20-session liquidity feature can be calculated?

Suggested answers

1. December 1 lies beyond verified coverage. The fixture supplies no closure evidence. Returning a closed-day result would turn missing knowledge into a false calendar assertion.
2. Yes. The decision checks that session's clock. History needs earlier covered sessions; the first session has none. Keep these checks separate and fail explicitly when a feature requires history.
3. The half-open core predicate returns False at the exact close. It does not model auctions or fill prices. A completed daily bar can still carry the close timestamp and become available later.
4. An aware datetime identifies an instant, not a universal calendar date. Convert through UTC into New York before identifying its session. A +14:00 representation can display the next local date.
5. No. The calendar establishes the scheduled window only. Every required value must exist, belong to the correct session and pass publication/receipt, processing-latency and any staleness checks.